A3-PS006-TC329

PS-006: Policy-Constrained Service Failover Demonstrator

Ankur Pratap Singh

Team Algo-Ancesstors

AIORI Hackathon 3

October 13, 2026

Problem Description

When a server goes down, where should the traffic go?

Most automatic failover tools only ask one thing: is the other server reachable? If yes, traffic moves there. That is fast, but it is also careless.

A replica in the wrong country can break data residency rules. A replica running old TLS can quietly weaken encryption. The outage ends, but a new problem starts.

Blind rerouting

Any reachable replica gets the traffic.

Residency risk

Data can cross a border without anyone deciding it should.

Encryption downgrade

A weaker TLS version can be picked up silently.

Background

Failover was built for uptime, not for rules

Cloud failover has one job: keep the service alive. Health checks spot a dead server and traffic shifts to whatever still answers. Rules about where data may live, or how strongly it must be encrypted, are usually someone else's problem [1].

Outage

Process crash, link loss or overload

→

Auto failover

Switch to any replica that responds

→

Gap

Location and TLS are never checked

Our job: close that gap, inside a lab, with simple and checkable rules.

Literature & RFC Review

Three standards shape our rules

RFC 6973 [1]

Privacy considerations for Internet protocols. Why we treat location and logs with care.

RFC 8446 [2]

TLS 1.3. Our reference for what a strong, modern handshake looks like.

RFC 9325 [3]

Recommendations for secure use of TLS and DTLS. It backs our baseline: no TLS below 1.2.

We use these as a guide for the policy file. We are not building a legal compliance engine.

Proposed Solution

A controller that asks before it switches

Clients talk only to our controller, which acts as a reverse proxy in front of two lab replicas. When the active one fails, the controller does not jump. It checks the other replica against a plain policy file first.

Health Monitor

Checks both replicas every 100-500 ms and spots the failure.

Policy Engine

Reads the replica's location label, runs a TLS handshake, and says pass or fail with a reason.

Router

Switches traffic only on a pass. On a fail it refuses and raises an alert.

policy.yaml

```
allowed_locations: [IN]
min_tls_version: "1.2"
logging: minimal
# timestamp, event,
replica, decision
```

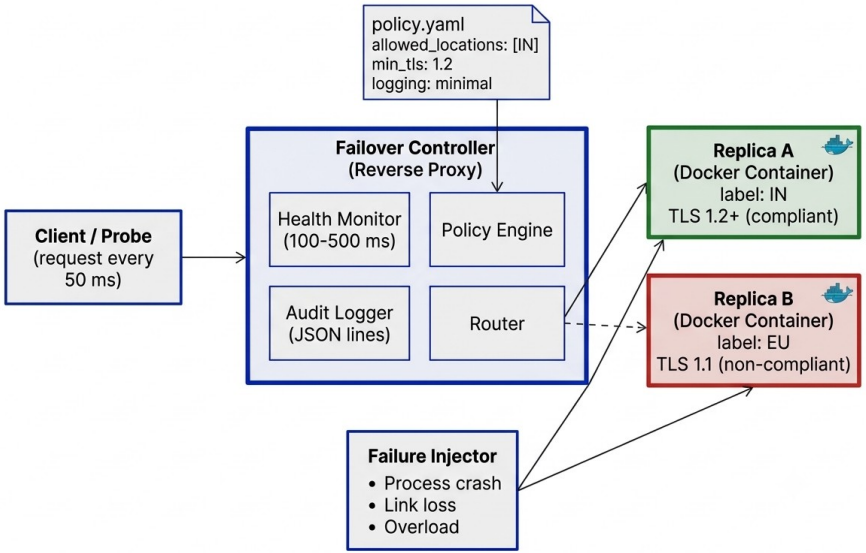
Optimization Proposed

What changes compared with ordinary failover

Aspect	Before: ordinary failover	After: our controller
Target choice	Any reachable replica	Only a replica that passes policy
Location	Ignored	Label must be in allowed_locations
TLS	Whatever connects	Handshake must meet the 1.2 baseline
Decision	Opaque	Deterministic: pass or fail, with a reason
Evidence	Scattered logs	Timestamped JSON audit timeline
Downtime	Rarely measured	Measured every run with a 50 ms probe

Solution Architecture & Design

Where each piece sits



1 **Controller is the front door**

Clients never reach a replica directly, so the switch is fully under our control.

2 **Replica B is wrong on purpose**

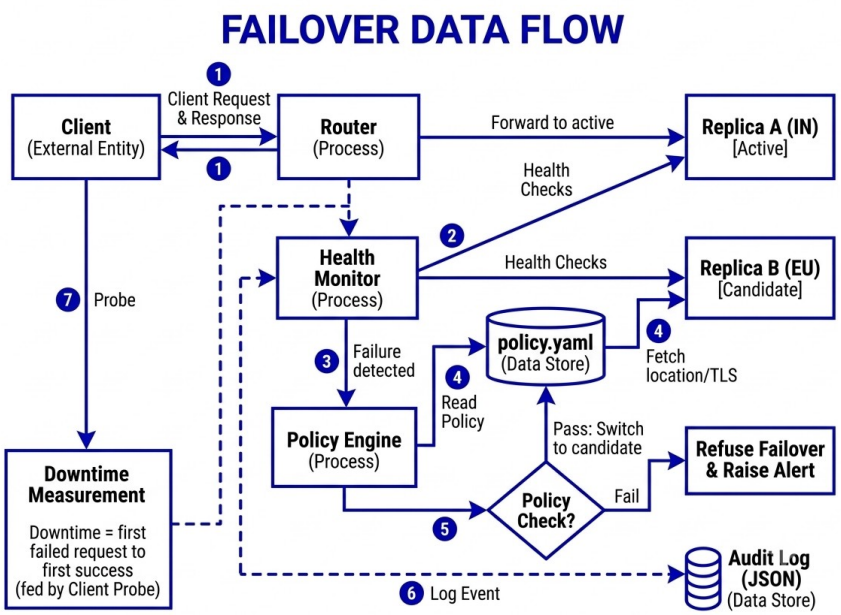
EU label and TLS 1.1, so there is always something to reject.

3 **Failure injector**

Crash, link loss and overload, each from a repeatable script.

Data Flow Diagram

How a failure turns into a decision



1 Detect

The health monitor flags the active replica as failed (steps 2-3).

2 Decide

The policy engine reads policy.yaml and checks the candidate's label and TLS version (steps 4-5).

3 Act and record

Pass: switch. Fail: refuse and alert. Every event goes to the audit log (step 6), and the probe measures downtime (step 7).

Deployment Timeline

Build one scenario end to end, then widen

1

Replicas

Two Docker replicas, IN and EU labels, own TLS certs

2

Controller

Proxy, health monitor, policy engine, router

3

Crash first

Full flow for the crash scenario, with downtime

4

More faults

Link loss and overload as variations

5

Demo + report

Audit timeline, rejection run, PDF, pseudocode

Demo plan

Crash, compliant target	Link loss, compliant target	Overload, non-compliant
Failover succeeds	Failover succeeds	Failover refused

Bibliography

[1] C. Cooper et al. "Privacy Considerations for Internet Protocols." RFC 6973, IETF, 2013.

[2] E. Rescorla. "The Transport Layer Security (TLS) Protocol Version 1.3." RFC 8446, IETF, 2018.

[3] Y. Sheffer, P. Saint-Andre, T. Fossati. "Recommendations for Secure Use of Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS)." RFC 9325, IETF, 2022.



Thank You for Your Attention!

Ankur Pratap Singh • Team Algo-Ancesstors • A3-PS006-TC329